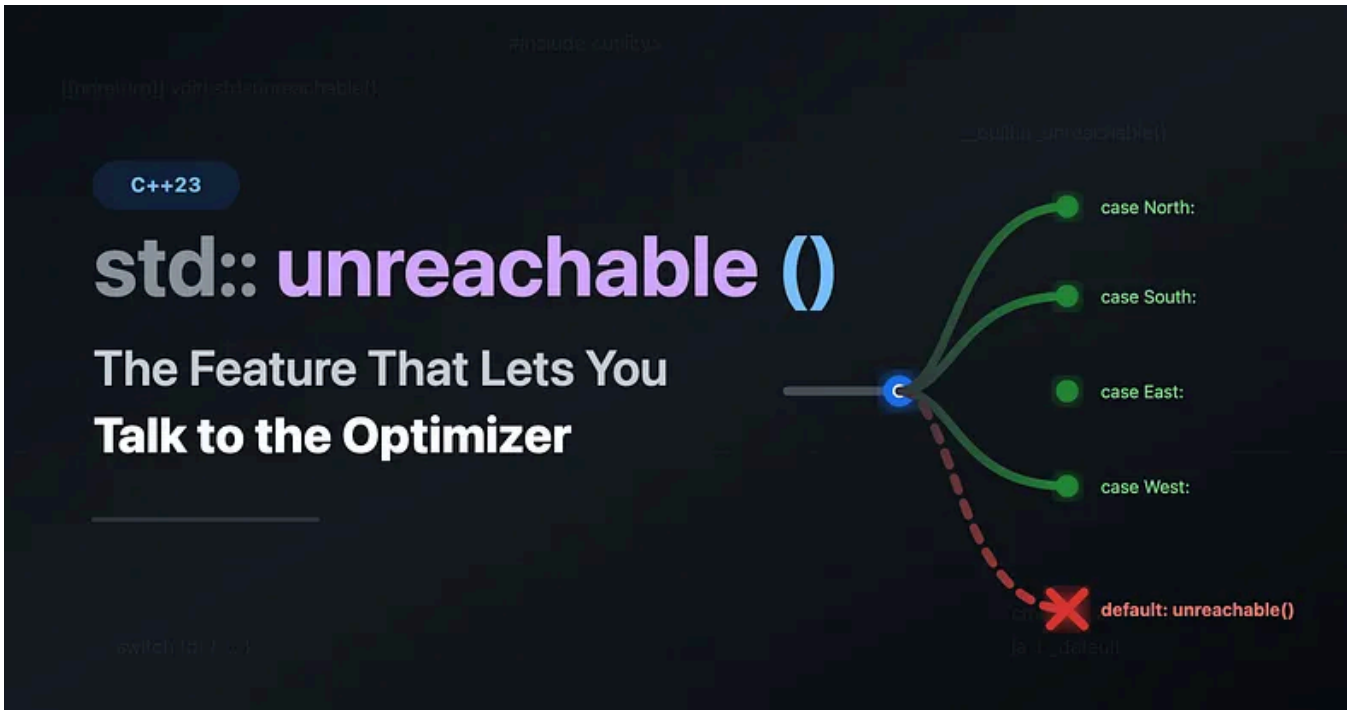
C++23 `std::unreachable()` — The Feature That Lets You Talk to the Optimizer

`C++23's std::unreachable()` lets you talk directly to the optimizer. See the assembly proof of how it speeds up code—and the fatal UB traps to avoid.

9 min read · 22 hours ago

Sagar Following

Listen Share More



I've been writing C++ for over 15 years, and every once in a while a new feature comes along that makes me think, "Finally." **`std::unreachable()`** is one of those features. It's small. It's simple. And it can genuinely change how the compiler optimizes your code — if you use it right.

Let me walk you through it.

Let's Understand First What Problem Are We Actually Solving?

You've written code like this. I know you have, because I have too:

```
enum class Direction { North, South, East, West };

std::string_view to_string(Direction d) {
    switch (d) {
        case Direction::North: return "North";
        case Direction::South: return "South";
        case Direction::East:  return "East";
        case Direction::West:  return "West";
    }
    // What goes here?
    return ""; // <-- this line bugs me
}
```

That `return ""` at the bottom? **It's dead code**. You know it's dead code. I know it's dead code. But the compiler doesn't. It sees a function with a return type, and it sees a code path that might fall through without returning. So it either:

1. Warns you about it (`-Wreturn-type`), or
2. Generates extra instructions to handle that "impossible" path.

Neither is great. We're lying to the compiler, and it's generating worse code because of it.

C++23 introduced `std::unreachable()` for to exactly address this problem.

C++23 `std::unreachable()`

Defined in `<utility>` , available in C++23:

```
[[noreturn]] void std::unreachable();
```

That's the whole signature. It tells the compiler: "*This point in the code will never be reached. Period. If it is, the behavior is undefined.*"

Here's our fixed function:

```
#include <utility>

enum class Direction { North, South, East, West };

std::string_view to_string(Direction d) {
    switch (d) {
        case Direction::North: return "North";
        case Direction::South: return "South";
        case Direction::East:  return "East";
        case Direction::West:  return "West";
    }
    std::unreachable();
}
```

No more bogus return. No more warnings. And the compiler can now assume that the switch always matches, which unlocks real optimizations.

Great! But whats happening under the hood here ?
Let me be blunt: `std::unreachable()` is a portable wrapper around compiler-specific intrinsics that have existed for years.

COMPILER	OLD WAY	C++23 WAY
GCC	`__builtin_unreachable()`	`std::unreachable()`
Clang	`__builtin_unreachable()`	`std::unreachable()`
MSVC	`__assume(false)`	`std::unreachable()`

If you've used any of these before, `std::unreachable()` does the exact same thing. The win is portability and readability. No more `#ifdef` drama.

Let's see the Optimization Difference

This is where it gets fun. Let's look at a real example.

```
int square_of_direction(Direction d) {
    int val;
    switch (d) {
        case Direction::North: val = 1; break;
        case Direction::South: val = 2; break;
        case Direction::East:  val = 3; break;
        case Direction::West:  val = 4; break;
        default: val = 0; break; // "just in case"
    }
    return val * val;
}
```

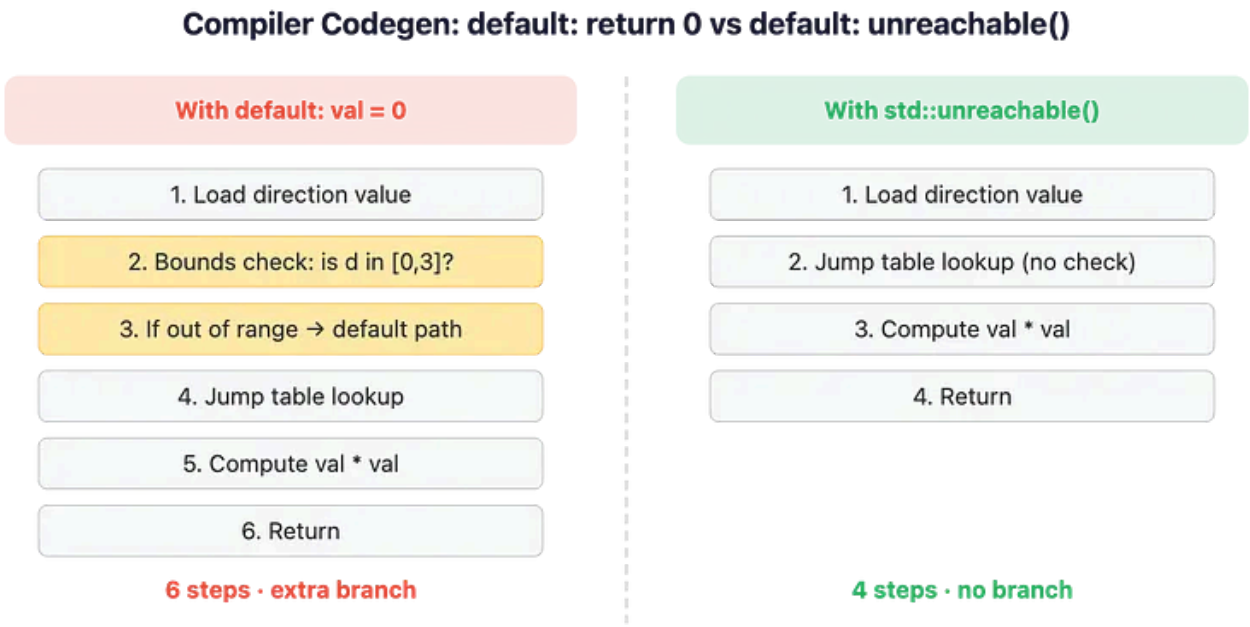
That defensive `default` case forces the compiler to emit bounds-checking code. It has to handle the possibility that `d` is some rogue integer.

Now compare:

```
int square_of_direction(Direction d) {
    int val;
    switch (d) {
        case Direction::North: val = 1; break;
        case Direction::South: val = 2; break;
        case Direction::East:  val = 3; break;
        case Direction::West:  val = 4; break;
        default: std::unreachable();
    }
    return val * val;
}
```

The compiler **now knows only** four values are possible. It can use a simple jump table with no bounds check. On hot paths, this matters.

Here's a visual of what happens in the compiler's mind:



That eliminated branch can be the difference between a cache-friendly hot loop and one that stalls.

Proof in Assembly: What the Compiler Actually Emits?

Diagrams are nice, but let's look at what **GCC 13.2** actually generates. I ran both versions through Compiler Explorer (`-O2 -std=c++23`) so you don't have to trust my word for it.

Here's a simpler example to keep the assembly readable:

```
enum class Color { Red, Green, Blue };

int color_to_int_defensive(Color c) {
    switch (c) {
        case Color::Red:   return 1;
        case Color::Green: return 2;
        case Color::Blue:  return 3;
        default:           return 0;  // defensive
    }
}

int color_to_int_unreachable(Color c) {
    switch (c) {
        case Color::Red:   return 1;
        case Color::Green: return 2;
```

```
        case Color::Blue:  return 3;
    }
    std::unreachable();
}
```

With defensive **`default: return 0`** (GCC 13.2, **`-O2`**):

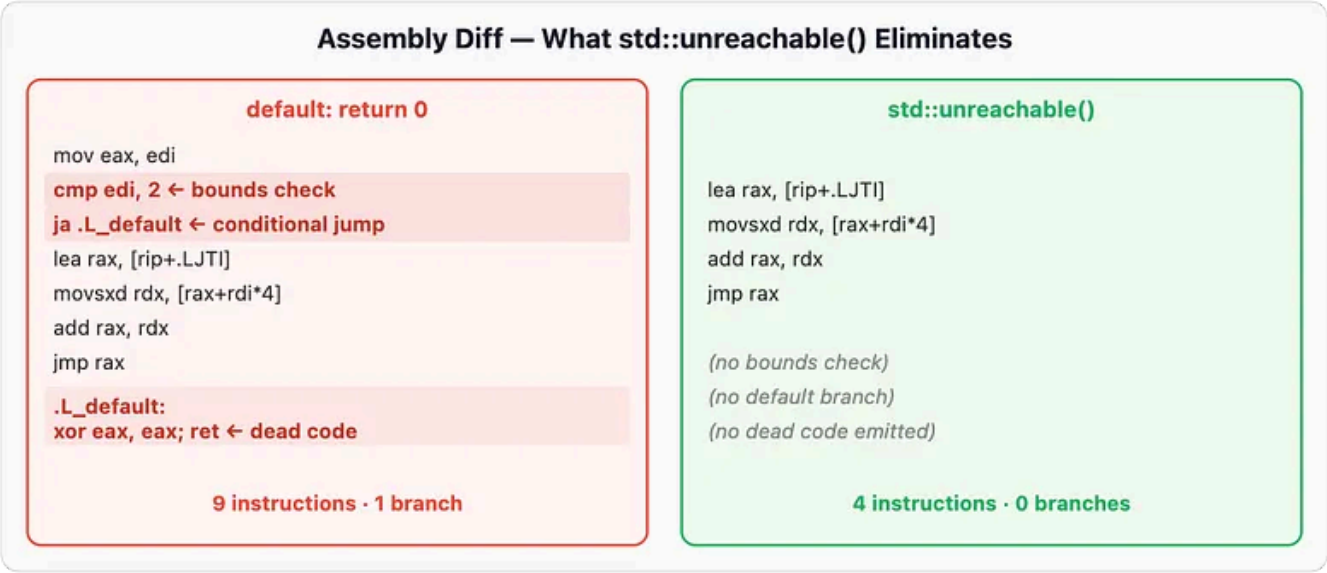
```
color_to_int_defensive(Color):
        mov     eax, edi            ; load enum value
        cmp     edi, 2              ; is it > 2?
        ja      .L_default          ; if so, jump to default
        lea     rax, [rip+.LJTI]    ; load jump table address
        movsxd  rdx, DWORD PTR [rax+rdi*4]
        add     rax, rdx
        jmp     rax                 ; indirect jump through table
.L_default:
        xor     eax, eax            ; return 0
        ret
.L_red:
        mov     eax, 1
        ret
.L_green:
        mov     eax, 2
        ret
.L_blue:
        mov     eax, 3
        ret
```

With **`std::unreachable()`** (GCC 13.2, **`-O2`**):

```
color_to_int_unreachable(Color):
        lea     rax, [rip+.LJTI]    ; load jump table address
        movsxd  rdx, DWORD PTR [rax+rdi*4]
        add     rax, rdx
        jmp     rax                 ; indirect jump through table
.L_red:
        mov     eax, 1
        ret
.L_green:
        mov     eax, 2
        ret
.L_blue:
```

```
        mov     eax, 3
        ret
```

See the difference? Let me highlight it:



The `cmp` + `ja` pair is gone. The entire `.L_default` block is gone. That's 5 fewer instructions and one fewer branch the CPU has to deal with. In a tight loop processing millions of enum values, that branch predictor pressure adds up.

You can verify all of this yourself on **Compiler Explorer** — just paste the code, pick GCC 13+ or Clang 16+ with `-O2 -std=c++23`, and watch the assembly shrink.

Now Here are some more practical use cases.

More Practical Use Cases

1. Unreachable `default` in Fully-Covered Switches

This is the bread and butter:

```
int get_priority(LogLevel level) {
    switch (level) {
        case LogLevel::Debug:   return 0;
        case LogLevel::Info:    return 1;
        case LogLevel::Warning: return 2;
        case LogLevel::Error:   return 3;
        case LogLevel::Fatal:   return 4;
    }
    std::unreachable();
}
```

2. After Infinite Loops That Genuinely Never Exit

```
[[noreturn]] void event_loop() {
    while (true) {
        auto event = wait_for_event();
        dispatch(event);
        // This loop never breaks. Ever. It's a daemon.
    }
    std::unreachable(); // Helps static analyzers and the compiler
}
```

3. Guarding Post-Condition Violations

```
int clamp_and_categorize(int value) {
    value = std::clamp(value, 0, 100);

    if (value < 50)  return 0;
    if (value <= 100) return 1;

    // value cannot be > 100 after clamp
    std::unreachable();
}
```

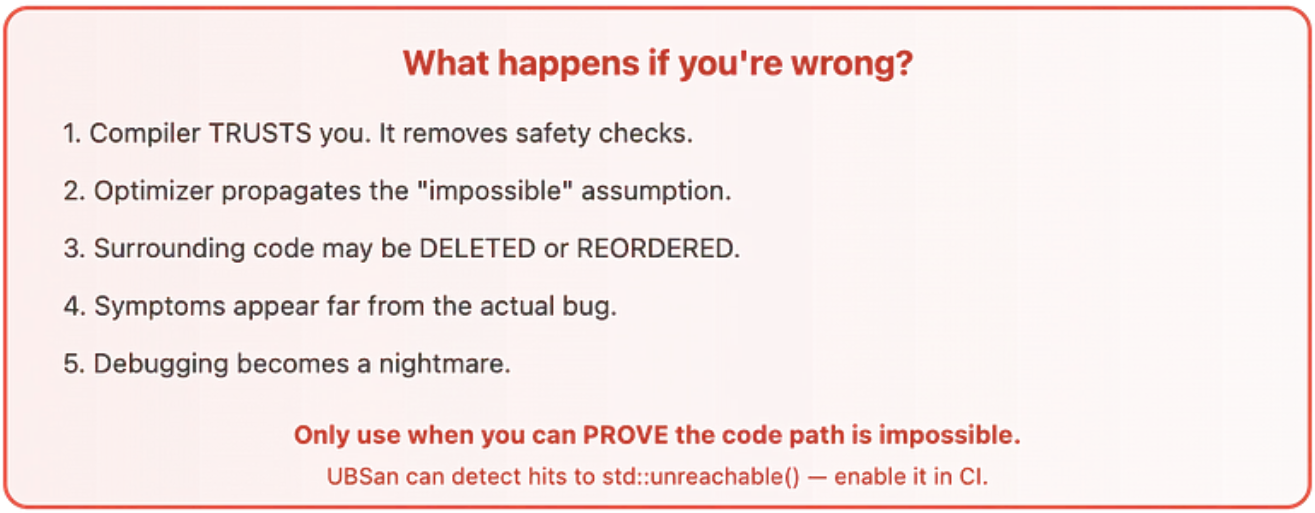
4. Trimming Impossible Branches in Performance-Critical Code

```
// You KNOW that 'index' has been validated upstream
float fast_lookup(const std::array<float, 4>& table, unsigned index) {
    if (index < table.size()) [[likely]] {
        return table[index];
    }
    std::unreachable();
}
```

This tells the compiler it can skip generating the “what if `index >= 4`?" fallback entirely.

The Danger Zone

I need to be very clear about this: **if execution ever actually reaches `std::unreachable()`, your program has undefined behavior.** Not "it returns garbage." Not "it throws an exception." Full, anything-can-happen, nasal-demons UB.



Here’s a classic example:

```
// DON'T DO THIS
int get_value(int selector) {
    switch (selector) {
        case 0: return 10;
        case 1: return 20;
        case 2: return 30;
        default: std::unreachable(); // What if selector == 7?
```

```
    }
}
```

If someone calls `get_value(7)`, you're done. The compiler might have already eliminated the bounds check, and now you're executing whatever garbage is at that jump table offset.

A Subtle But Critical Edge Case: Enums and External Input

The `Direction` and `LogLevel` examples above work because those are `enum class` types that we own and we exhaustively cover. That's the ideal scenario.

But consider this: what if `Direction` were a plain C-style enum being parsed from a network packet, a config file, or a third-party library?

```
// ⚠Direction comes from a network protocol
// The wire could send literally any byte value
enum Direction : uint8_t { North = 0, South = 1, East = 2, West = 3 };

std::string_view to_string(Direction d) {
    switch (d) {
        case North: return "North";
        case South: return "South";
        case East:  return "East";
        case West:  return "West";
        default:
            // std::unreachable() would be WRONG here
            // Handle the bad input explicitly
            log_fatal("Invalid Direction from wire: {}", static_cast<int>(d));
            return "Unknown";
    }
}
```

The rule is simple:
if the value can come from anywhere you don't control — user input, deserialized data, a plugin API, a cast from integer — then you **cannot** prove the `default` is unreachable. Keep the error-handling branch. The minor optimization win is not worth the UB risk.

`std::unreachable()` is for invariants you can prove from the structure of your own code, not for assumptions about what the outside world might hand you.

A Safer Pattern for Debug Builds

Here's what I actually do in production. I wrap it:

```
#include <utility>
#include <cassert>

[[noreturn]] inline void truly_unreachable(
    [[maybe_unused]] const char* msg = "unreachable code reached"
) {
    #ifndef NDEBUG
        // Debug: crash loudly with a message
        assert(false && "unreachable code was reached");
        // If assert is somehow disabled, abort anyway
        std::abort();
    #else
        // Release: trust the programmer, optimize aggressively
        std::unreachable();
    #endif
}
```

Usage:

```
std::string_view to_string(Direction d) {
    switch (d) {
        case Direction::North: return "North";
        case Direction::South: return "South";
        case Direction::East:  return "East";
        case Direction::West:  return "West";
    }
    truly_unreachable("invalid Direction value");
}
```

Now you get **hard crashes in debug** and **full optimization in release**. Best of both worlds.

It's also worth noting: many UB sanitizers (`-fsanitize=undefined` on GCC/Clang) can detect execution reaching `std::unreachable()` at runtime in instrumented builds. If you're running **UBSan** in CI — and you should be — it'll catch these hits before they ever make it to production. That makes the debug-wrapper + sanitizer combo a

How It Compares to Other Approaches?

Let me lay out all the alternatives I've used over the years and why `std::unreachable()` is the right tool now:

```
// Throws — adds overhead, function can't be noexcept
throw std::logic_error("unreachable");

// Abort — prevents optimizations, still generates the code path
std::abort();

// __builtin_unreachable() — GCC/Clang only
__builtin_unreachable();

// __assume(false) — MSVC only
__assume(false);

// Empty default — compiler warning, missed optimizations
default: break;

// GOOD: std::unreachable() — portable, standard, optimizable
std::unreachable();
```

Compiler Support (as of 2026)

You're probably fine as major compilers already support this:

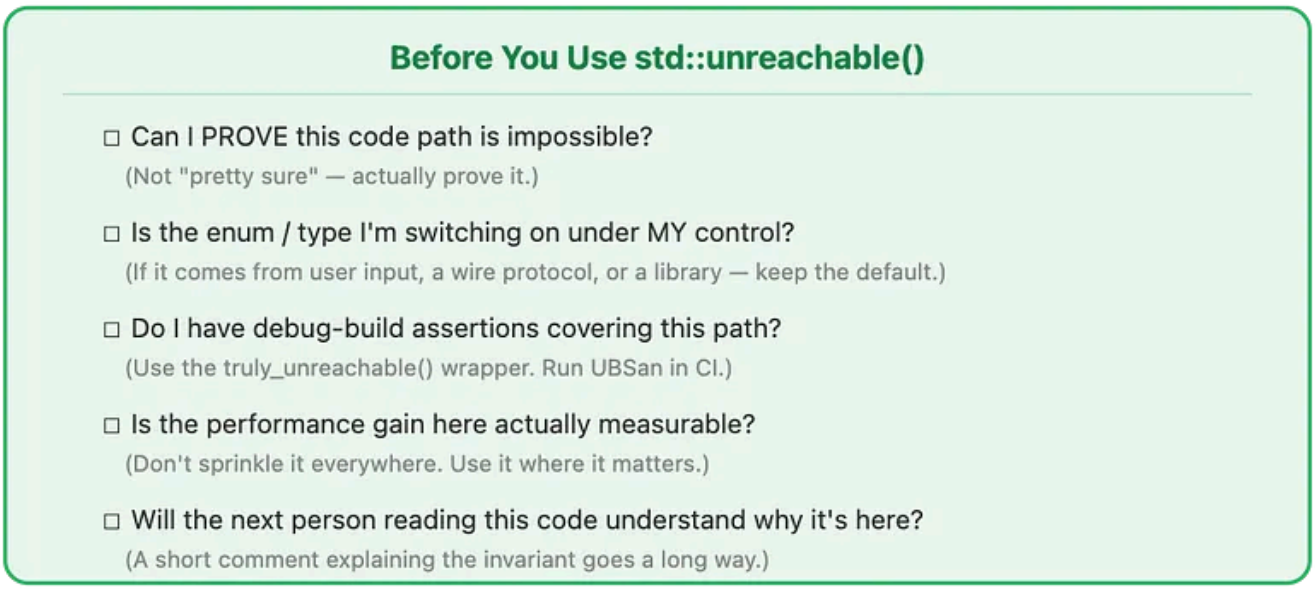
COMPILER	MINIMUM VERSION	FLAG NEEDED
GCC	12.1	`-std=c++23`
Clang	15.0	`-std=c++23`
MSVC	19.34 (VS 2022 17.4)	`/std:c++latest` or `/std:c++23`

If you're stuck on an older standard, here's a polyfill:

```
#if __cplusplus >= 202302L
    #include <utility>
    #define MY_UNREACHABLE() std::unreachable()
#elif defined(__GNUC__) || defined(__clang__)
    #define MY_UNREACHABLE() __builtin_unreachable()
#elif defined(_MSC_VER)
    #define MY_UNREACHABLE() __assume(false)
#else
    #define MY_UNREACHABLE() do { } while(0) // fallback: no-op
#endif
```

Rules of Thumb

After using this in production for a while, here's my personal checklist before I type `std::unreachable()`:



Ok we have seen usage, benefits etc etc but its also worth knowing when not to use `std::unreachable()` .

When NOT to Use It?

This is important enough to call out explicitly.

Don't sprinkle **`std::unreachable()`** into every **`default:`** clause. It's tempting once you learn about it — I get it, I did the same thing. But ask yourself: ***where does this value come from?***

- **User input?** Keep the error-handling branch.
- **Deserialized data from disk or network?** Keep the error-handling branch.
- **A third-party library's enum that might add variants in the next release?** Keep the error-handling branch.
- **A `static_cast<MyEnum>(some_integer)` ?** Keep the error-handling branch.
- **Your own closed `enum class` with all cases covered in the same translation unit?** *Now* you can reach for `std::unreachable()` .

The performance difference between a branch-with-error-handling and **`std::unreachable()`** is almost never measurable in code that deals with I/O or external data. Save the sharp tool for the tight inner loops where it actually matters.